

## Assignment: Build a Mini Hindi-GPT From Scratch

### Instructions

1. Please see the course policy on extension days and bonus points. There will be no deviation from the policy for any late submissions.
2. Only one member of the group should submit the assignment as a ZIP file on Classroom.
3. All members of the group will be expected to be familiar with all parts of the assignment for the demo.
4. Doubts will be resolved only through the Google Form shared on Classroom.
5. The entire assignment must be completed collectively by the group. Do not divide the workload by assigning specific tasks to individual members, as the workload

### Datasets

You will be using two datasets for this assignment. They have been provided via the following Google Drive links: **dataset link**

- **Hindi corpus**
- **Text Classification Dataset**

### Task 1: Data Preprocessing (25)

Before building the model, you must prepare the data pipelines for both tasks. Implement the preprocessing modules as follows:

1. **Corpus Preprocessing:** Read the provided Hindi corpus dataset. Since you are only provided the raw dataset, you must manually split the corpus into a training set and a validation set. You must also implement logic to generate dataset pairs  $(x, y)$  for language modeling, where the target sequence  $y$  is the input sequence  $x$  shifted forward by one position.
2. **Vocabulary Generation:** Train a subword tokenizer on the training split of the Hindi corpus using Byte-Pair Encoding (BPE) or SentencePiece with a **target vocabulary size of 5,000**. Create encoding and decoding functions to convert Hindi strings to lists of integer token IDs and vice versa.
3. **Classification Preprocessing:** Read the provided text classification dataset. If not already partitioned, split this into a training and validation set. Map the labels into 3 distinct integer classes (0 = Negative, 1 = Neutral, 2 = Positive). Generate dataset pairs  $(x, y)$  where  $x$  represents the tokenized review sequence and  $y$  represents the corresponding integer class label.

## Task 2: Decoder-Only Transformer Architecture and Next Word Prediction (50)

Implement a simplified, **decoder-only** Transformer similar to the GPT-2 architecture, and train it as an autoregressive language model on the Hindi corpus. You will build the core backbone once and reuse it for Task 3.

1. **Multi-Head Attention:** Implement query, key, and value projections, followed by scaled dot-product attention. Crucially, implement the **causal mask** (a lower-triangular matrix) to ensure tokens cannot attend to future tokens.
2. **FeedForward Network:** Implement a two-layer multi-layer perceptron (MLP) with a ReLU activation function.
3. **Transformer Block:** Combine the masked multi-head attention, feedforward network, and layer normalization, connected via residual (skip) connections.
4. **MiniGPT Base:** Combine token embeddings, positional embeddings, and a sequential stack of Transformer blocks.
5. **Language Modeling Head:** Implement the `GPTLanguageModel` class, which attaches a linear layer on top of the MiniGPT backbone mapping from the embedding dimension to the vocabulary size.
6. **Language Model Training:** Train the model on the training split of the Hindi corpus using Cross-Entropy Loss.
7. **Language Model Evaluation:** Calculate and report the validation Perplexity. Write a generation function to autoregressively sample from the model and generate 50 tokens of Hindi text .

## Task 3: Text Classification (25)

Attach a classification head to the **same** GPT backbone trained in Task 2, and train it on the classification dataset.

1. **Classification Head:** Implement the `GPTClassifier` class. Attach a linear classification head mapping to the 3 output classes.
2. **Sequence Alignment:** Because this is a causal, decoder-only model, you **must** extract the features of the **last** token in the sequence to make a classification decision.
3. **Classifier Training & Evaluation:** Train the model on the classification training split. Report the final classification accuracy on the validation split.

## Deliverables

- **Code**
- **Model Weights:** The saved model checkpoints (weights) for both the Language Model (Task 2) and the Classifier (Task 3).
- **Outputs:** A generated text file containing the model's text generation output (Task 2) and a sample of 5 correctly predicted movie reviews (Task 3).

## Report

Provide a short report (1-2 pages) containing:

- state the work distribution among group members.
- **Plots & Metrics:** The training and validation loss curves for both tasks. A final report of Perplexity (Task 2) and Accuracy (Task 3).
- Any other detail you wish to put.